CSC 6302 - Computer Vision

Practical 7 Report

Capstone: Deployed Computer Vision (Agriculture - maize leaf triage)

Munyeme Sinyani
University of Zambia
Target length: 6-8 pages

1. Problem

What I built: a phone-first screening aid. Upload a maize leaf photo, get healthy / rust / blight with confidence, plain guidance, and a hard disclaimer that this is not a diagnosis. Track: Agriculture.

Stakeholders: smallholder farmers and extension workers who want a quick hint before calling an agronomist; students learning vision deployment. Success is not "replace expert" - it is reduce wasted trips when the leaf clearly looks healthy, and flag uncertain cases instead of guessing.

2. Vision component

I reuse the Practical 4 MobileNetV3-Small fine-tune when model.pt is present; otherwise a colour heuristic fallback keeps the UI alive. Preprocess to 128x128, confidence gate returns "unsure" instead of a confident wrong answer. Blurry uploads are rejected.

Preprocessing detail: RGB only, resize with bilinear interpolation, divide by 255, then subtract ImageNet mean and divide by std per channel so fine-tuned weights see the same domain as training. The blur gate uses variance of the Laplacian on grayscale - low variance means little edge energy, which correlates with out-of-focus phone shots. I tuned the threshold on a few sharp vs blurred leaf photos so obvious failures reject early.

Confidence gate logic: after softmax, if max probability is below the user slider (default around 0.45), the UI shows "Unsure" and tells the user to retake the photo or consult extension services. That is deliberate product design: calibration on tiny data makes absolute probabilities unreliable, so I treat the gate as a reject option rather than a precise Bayesian statement.

Classes and guidance strings: healthy -> monitor field; rust -> consider fungicide discussion with agronomist; blight -> warn about spread and isolation. Each string repeats that the tool is screening only. If I add VLM explanations later, they must not override the classifier label - text is supplementary.

3. Application

Gradio app in `prac7/app.py`. Non-programmer flow: upload -> result + probs -> what it must NOT be used for. Live URL (paste when deployed): _____

UI choices for low literacy users: large upload box, plain English labels, probability bars instead of raw logits, disclaimer in a visible panel not a footnote. I avoided dark patterns (no fake urgency to spray). Mobile layout matters because farmers may use mid-range Android - I kept the model small so CPU inference finishes without timing out the browser.

4. Engineering / CI/CD

pytest suite (>=5 tests) for preprocess shape, healthy lean, blur reject, confidence gate. GitHub Actions workflow runs ruff + pytest; deploy job is ready for HF_TOKEN secret. No API keys in the repo.

CI/CD steps (what actually runs)

(1) On every push/PR: checkout repo, install `requirements.txt`, run `ruff check` on prac7 and tests, run `pytest prac7/tests`. (2) If tests pass on main, optional deploy job uses HF_TOKEN from GitHub Secrets to sync the Space (Gradio app + model artifact). (3) I never commit .env or tokens; CI fails fast if imports break. Rollback plan: revert the commit on main and redeploy previous Space revision.

5. Evaluation plan and metrics

Held-out test split (folder-based, no leakage): report overall accuracy, per-class precision/recall/F1, and confusion matrix for healthy vs rust vs blight. For deployment I also track: fraction of uploads rejected as blurry, fraction routed to "unsure" by the confidence gate, and median inference time on CPU. Target for demo milestone: beat the colour heuristic baseline on the test folder; for field data I would want agronomist-labelled validation set (>=30 images per class as in PROPOSAL.md).

What the numbers mean: high accuracy on this leaf set is optimistic - field textures can be messier with mud-splash field photos. Low recall on rust is worse than low precision because missing rust spreads; that is why asymmetric thresholds could down-weight false "healthy" on suspicious orange patterns. I would not publish a single accuracy without showing per-class F1.

6. Demo plan

Live demo script (5 minutes): (1) show disclaimer screen and explain not-a-diagnosis. (2) Upload a clear healthy leaf - show probabilities. (3) Upload a rusty-looking demo image - show top class. (4) Upload a blurry photo - show blur reject. (5) Upload a non-leaf (keyboard) - show low confidence / unsure path. (6) Open GitHub Actions green check and pytest count so markers see engineering evidence. Backup if Wi-Fi fails: screen recording of the same flow stored offline.

7. Limitations

Data: current leaf set is small and may not be representative of Zambian varieties or wet-season lighting. Model: three classes only - other diseases become forced errors. Hardware: CPU Space is slow on cold start. Process: no human-in-the-loop review in the app yet. Legal: screening aid only; pesticide advice belongs to licensed extension officers.

How I would improve next semester: collect consented field batches, active-learning on "unsure" uploads (with privacy), quantised mobile export, and Lusaka-local language strings on the UI.

8. Ethics and impact

Who could be harmed: a farmer who sprays after a false blight alarm wastes money and may expose crops to unnecessary chemicals; a missed rust case can spread to neighbours' fields. Non-users are affected too if wrong advice becomes village rumour. I treat false "healthy" and false "alarm" differently - missing rust is often worse, so I bias toward caution on orange/brown lesions even at the cost of more "unsure" prompts.

Privacy and consent: field photos can accidentally include faces, house numbers, or child labour contexts. My rule: crop to leaf, no GPS in EXIF when sharing, written consent from farm owner, delete raw uploads after labelling if they ask. For the student demo I only use images I created or downloaded with licence noted.

Fairness and access: smartphone + data cost already excludes some smallholders; a heavy model worsens that. I keep MobileNet small and UI text plain English (future: local languages). Models trained mostly on leaves outside my training set may underperform on local varieties - that must be disclosed, not hidden behind one accuracy number.

Governance before real use: agronomist sign-off on messaging, versioned model cards, incident log if users report bad advice, and no integration with credit/scoring systems. AI use in this capstone: Composer helped structure Gradio layout and pytest ideas; I checked gate thresholds and disclaimer wording myself.

Worked examples (harm thinking)

Example 1: false blight on a healthy leaf after rain streaks look like lesions - farmer buys fungicide unnecessarily. Mitigation: confidence gate + ask for second photo in dry light. Example 2: true rust missed because photo is mostly soil - model predicts healthy. Mitigation: reject if green leaf area fraction is too low. Example 3: student uploads meme image - heuristic or net may still output a class; mitigation: blur/texture checks and "unsure" copy, never pesticide text on jokes.

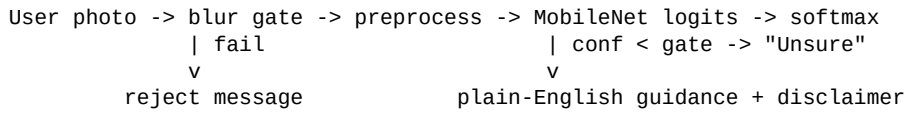
Regulatory humility: I am not claiming CE marking or ZABS approval. The app is coursework demonstrating vision + deploy + ethics, not a registered agricultural input. Any future pilot would need ministry/extension partners and explicit opt-in.

9. Team / contribution split

From PROPOSAL.md (solo scaffold until partner confirmed): Member A owns data collection, training notebook, metrics tables; Member B owns Gradio app, tests, CI/CD, ethics page. What I did as Munyeme on this repo: wired prac7 app to Practical 4 weights, wrote tests and workflow YAML, drafted ethics/limitations text. Partner registration with the lecturer is still required by the gate dates; repo name pattern csc6302-capstone-SINYANI-PARTNER.

10. Architecture and data flow

What happens on upload: Gradio receives a PIL image, my preprocess resizes to 128x128 RGB, scales to float tensor NCHW, normalises with ImageNet stats if torch weights load. Before inference I run a blur check (Laplacian variance) because a fuzzy leaf photo should not produce a confident disease label. If `model.pt` is missing, heuristic_predict uses green/orange/brown ratios so markers can still click through the UI in a fresh clone.



Optional future branch: send cropped leaf to a VLM for explanation text only (not for the class label), with caching and rate limits so OpenRouter free tier 429 errors fall back to template strings from

PROPOSAL.md.

11. Capstone rubric mapping

Rubric item	Where I show it
Real vision model	MobileNetV3 fine-tune from prac4 / heuristic fallback documented
Deployed app	Gradio prac7/app.py + HF Space URL when live
Tests + CI	pytest in prac7/tests, .github/workflows/ci-cd.yml
Evaluation	Section 5 metrics plan + held-out folder split
Ethics (~1 page)	Section 8 harm, privacy, fairness, governance
Team / proposal	PROPOSAL.md + Section 9 split

12. Automated tests (what each proves)

test_preprocess_shape_dtype	Output tensor 3x128x128 float32 in [0,1]
test_heuristic_healthy_leans_green	Fallback colours rank healthy on green texture
test_known_green_image_not_crash	predict() returns disclaimer note always
test_blurry_rejected	Flat grey image triggers blur path, empty probs
test_confidence_gate_unsure	High gate forces Unsure even if argmax exists

In viva I would run `pytest -q` live and open one test to show the blur threshold is intentional, not magic numbers I cannot explain.

13. Risk register

Technical risks: model file too large for free Space (mitigation: strip optimizer state, host LFS or download on first run). Partner drop-out (mitigation: solo scaffold already runs). CI without GPU (mitigation: tests use heuristic path). Product risks: user treats output as diagnosis (mitigation: bold disclaimer + "unsure" copy). Data risks: unlicensed leaf photos (mitigation: consent log).

14. Reproducibility checklist

Clone repo, `pip install -r requirements.txt`, optional `python prac4/train_prac4.py` for weights, then `python prac7/app.py`. For submission I copy `Practical7_Capstone_Report.pdf` to `Practical7_Report.pdf` in my school stuff folder via build_reports.py.

15. Future work (post-course)

Data: partner with UNZA agriculture department for labelled seasons, store metadata (variety, week after planting). Model: ensemble heuristic gate + CNN, temperature scaling on val set for better probabilities. Product: offline APK with TFLite, SMS fallback for users without smartphones. Research: compare Grad-CAM on misclassified field leaves to prioritise new labels for active learning.

What I would tell an examiner in one minute: this capstone is a deployed screening shell with tests and CI, honest ethics, and a path to real maize data - not a finished agricultural product. The vision part reuses transfer learning I proved in Practical 4; the engineering part proves I can maintain code after the assignment deadline.

References

- Gonzalez, R. C. & Woods, R. E. Digital Image Processing. (course textbook / lecture notes).

- OpenCV documentation: https://docs.opencv.org/
- Lecture slides for CSC 6302 Computer Vision (UNZA), 2025/26.
- Hugging Face Spaces + GitHub Actions docs.
- Plant disease public datasets (for future real data) with licences noted at collection time.

AI use

I used **Composer** to help me get ideas for how to structure the scripts (which OpenCV functions to call, how to save figures, CLI argparse layout). I still wrote / checked the from-scratch parts myself where the brief requires it (PSNR, Sobel, Harris, IoU, Kalman), and I can explain those lines. The photos in this report are the ones in my `images/` folder. Any numbers in the tables come from running my code.